Physics-Inspired Neural Networks with Learnable Gradient Metric for Spectral Feature Learning

Anonymous Authors[1]

Abstract

Spectral methods are widely used in machine learning for feature engineering and representation learning. Typically, such approaches rely on defining a kernel or an integral/differential operator whose eigenfunctions are then used to construct data-driven features. In practice, however, the performance of spectral methods is highly sensitive to the choice of kernel, feature scaling, and associated hyperparameters, often requiring substantial manual tuning.

In this work, we propose an alternative spectral framework that alleviates these limitations. Our approach is based on eigenfunctions of a modified Dirichlet energy functional, defined as the expected squared norm of the gradient of a function. The key modification consists in introducing a learnable linear transformation of the gradient within the Dirichlet energy, allowing the spectral structure to be adapted directly from data.

This formulation provides several important benefits. First, it relaxes the classical manifold hypothesis by interpolating between differential-geometric operators, such as the Laplace–Beltrami operator, and purely statistical objects defined by energy spectra. Second, it significantly reduces the need for manual hyperparameter tuning and feature scaling, as the relevant transformations are learned automatically. Finally, the proposed method yields compact and expressive data representations that are naturally aligned with downstream tasks, such as classification.

We evaluate EFDO on six datasets spanning two synthetic benchmarks, one tabular classification task, and three image-based problems: Two Moons, Circles, HTRU2, MNIST, and CIFAR-10 with pretrained ResNet-18 embeddings. On MNIST, the strongest identity-metric variant reaches 94.53% validation accuracy; on CIFAR-10 embeddings it reaches 85.50% validation accuracy under the same protocol. The rotation–scaling variant applies $\mathbf{A}(\mathbf{x})=\mathbf{\Lambda}(\mathbf{x})\mathbf{U}(\mathbf{x})$ to gradients as $\mathbf{v}\mapsto\mathbf{\Lambda}(\mathbf{x})\big(\mathbf{U}(\mathbf{x})\mathbf{v}\big)$ so both factors receive non-trivial gradients; see Sec. 2.4. NeuralEF (Deng et al., 2022) reports 82.52% MNIST *test* accuracy at $K=16$; our MNIST numbers are *validation* accuracy and therefore not directly comparable, but the gap is large enough to motivate further aligned evaluation. Logistic regression and random forests denote standard penalised linear and ensemble tree baselines with hyperparameters held fixed across replicates (Pedregosa et al., 2011). Training traces and eigenfunction plots illustrate convergence and geometry-sensitive coordinates.

1. Introduction

Contributions. The main contributions of this work are as follows:

- We introduce a learnable modification of the Dirichlet energy that generalizes classical spectral operators.
- We show that the resulting eigenfunctions provide compact and task-adaptive representations.
- We demonstrate empirically that the proposed approach reduces sensitivity to feature scaling and hyperparameter tuning.
- Empirical study on five benchmarks against random forests, logistic regression, and NeuralEF (Deng et al., 2022).
- Three learnable gradient metrics: identity OFF, diagonal scaling DIAG, and a Givens-chain rotation model TROTTER with the gradient application order fixed as in Sec. 2.4.

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the AI4Physics Workshop at the International Conference on Machine Learning (AI4Physics@ICML 2026). Do not distribute.

Graph-based spectral methods build a Laplacian on finite samples and use its eigenmaps as geometry-aware features (Gomez-Chova et al., 2008; Kunegis et al.). Their theory connects graph eigenfunctions to Laplace–Beltrami modes when data concentrate on a manifold (Belkin & Niyogi, 2008), which explains strong performance in clustering and semi-supervised learning (Ng et al., 2001). Two bottlenecks motivate mesh-free alternatives: eigencomputation cost grows with dataset size (Ford, 2015), and test-time evaluation at new points typically requires rebuilding the graph and its spectrum (Belkin & Niyogi, 2008).

Neural surrogates for operator eigenproblems are an active line of work (Jin et al., 2020; Deng et al., 2022; Choo et al., 2020). Automatic differentiation makes it feasible to minimise Rayleigh-type objectives with PDE-style regularity while remaining discretisation-free in the input domain (Feld et al., 2019; Lagaris et al., 1997).

We study *learnable-metric Dirichlet features* (EFDO): a sequence of scalar maps (ϕ_j) trained with a modified Dirichlet penalty that applies a data-dependent linear map $\mathbf{A}(\mathbf{x})$ to gradients (Evans, 2022), together with batch Gram orthogonality and cross-entropy on linear logits. The outer schedule freezes earlier coordinates and updates one map at a time, yielding cheap inference once training finishes. Section 2 derives the losses and weighting; Section 3 reports benchmarks and visualisations; Section 4 summarises limitations. We briefly discuss overfitting on synthetic examples where reference spectra are known; aggressive data augmentation is left for future work.

2. Methodology.

In the present section we describe the key ideas of the approach introduced. We first derive the loss as a weighted sum of three terms, then the dynamic weights, then the learnable gradient metric $\mathbf{A}(\mathbf{x})$. An optional graph-Laplacian warm-start could set auxiliary targets but is *not* used in the experiments below.

2.1. Loss-Function.

In the present work we follow the classical approach for computing eigenfunctions through minimisation of the Rayleigh quotient subject to orthogonality conditions. To this end, we consider a probability distribution with density $p(\mathbf{x})$. This probability distribution is utilised to introduce the following notion of the inner product:

$$\langle f_1, f_2 \rangle_0 = \int f_1(\mathbf{x}) f_2(\mathbf{x}) p(\mathbf{x}) d\mathbf{x} \tag{1}$$

Here f_1 and f_2 are functions with finite second moments.

Alternative product or bilinear form can be naturally introduced:

$$\langle f_1, f_2 \rangle_{\text{de}} = \int \sum_{a=1}^{d} \frac{\partial f_1(\mathbf{x})}{\partial x_a} \frac{\partial f_2(\mathbf{x})}{\partial x_a} p(\mathbf{x})\, \mathrm{d}\mathbf{x} \tag{2}$$

Here d is the dimension of $\mathbf{x}$.

This product becomes a regular Dirichlet Energy for $f_1 = f_2$:

$$\begin{aligned} \langle f, f \rangle_{\text{de}} &= \int \left(\sum_{a=1}^{d} \frac{\partial f(\mathbf{x})}{\partial x_a} \frac{\partial f(\mathbf{x})}{\partial x_a} \right) p(\mathbf{x})\, \mathrm{d}\mathbf{x} \\ &= \int \|\nabla f(\mathbf{x})\|^2 p(\mathbf{x})\, \mathrm{d}\mathbf{x} \end{aligned} \tag{3}$$

In the present work we introduce a learnable matrix $\mathbf{A}(\mathbf{x})$ to modify the Dirichlet energy and the corresponding bilinear form:

$$\langle f_1, f_2 \rangle_{\text{mde}} = \int \sum_{a_1,a_2,b_1,b_2=1}^{d} A_{a_1 b_1} A_{a_2 b_2} \frac{\partial f_1(\mathbf{x})}{\partial x_{b_1}} \frac{\partial f_2(\mathbf{x})}{\partial x_{b_2}} p(\mathbf{x})\, \mathrm{d}\mathbf{x} \tag{4}$$

The Modified Dirichlet Energy (MDE) can be computed as follows:

$$\langle f, f \rangle_{\text{mde}} = \int \left\| \mathbf{A}(\mathbf{x}) \nabla f(\mathbf{x}) \right\|^2 p(\mathbf{x})\, \mathrm{d}\mathbf{x} \tag{5}$$

The idea is to compute K coordinate maps $\phi_1(\mathbf{x}), \ldots, \phi_K(\mathbf{x})$ via Rayleigh-ratio minimisation under an orthonormality constraint. Matrix $\mathbf{A}(\mathbf{x})$ is learnt jointly with the basis so that a linear classifier atop $(\phi_1, \ldots, \phi_K)$ obtains high accuracy.

In other words, we have a loss-function that corresponds to the MDE:

$$\mathcal{L}_{\text{mde}} = \int \left\| \mathbf{A}(\mathbf{x}) \nabla \phi_k(\mathbf{x}) \right\|^2 p(\mathbf{x})\, \mathrm{d}\mathbf{x} \tag{6}$$

where ϕ_k is the currently active eigenfunction at outer index k during the sequential training schedule. There is a loss-function that enforces orthonormality constraint:

$$\mathcal{L}_{\text{orth}} = \sum_{\alpha,\beta=1}^{K} \left(\int \left(\phi_\alpha(\mathbf{x}) \phi_\beta(\mathbf{x}) - \delta_{\alpha\beta} \right) p(\mathbf{x}) d\mathbf{x} \right)^2 \tag{7}$$

Here $\delta_{\alpha\beta}$ is the Kronecker symbol.

Remark 2.1 (Batch Gram surrogate). In optimisation we minimise $\|\hat{C}_k - I_k\|_F^2$, where $\hat{C}_k = \frac{1}{B}\Phi_{1:k}^\top \Phi_{1:k}$. Thus the finite-sample Gram surrogate departs from (7) but coincides with the Frobenius penalty on $\hat{C}_k$ minimised throughout optimisation.

Features $\phi_1(\mathbf{x}), \ldots, \phi_K(\mathbf{x})$ are utilised to train the classifier:

$$\log\big(p(l|\mathbf{x})\big) \propto B_l + \sum_{\gamma=1}^{K} W_{l\gamma}\phi_\gamma(\mathbf{x}) \tag{8}$$

Here B is the bias, W stores the weights, $\log(p(l|\mathbf{x}))$ denotes the logits. Weights and bias are learnt from cross-entropy loss-function $\mathcal{L}_{\text{class}}$ minimisation.

Finally, all three loss-functions are combined together via weighting:

$$\mathcal{L} = \text{sg}(w_{\text{orth}})\mathcal{L}_{\text{orth}} + \text{sg}(w_{\text{class}})\mathcal{L}_{\text{class}} + \text{sg}(w_{\text{mde}})\mathcal{L}_{\text{mde}} \tag{9}$$

Here sg is a stop-gradient operation that means that derivatives of weights should not be computed in the back-propagation step.

It is worth noting that integrals over the probability distribution is utilised in the loss-functions calculation as it follows from Eq. (6) and Eq. (7). In practice, the density of probability distribution is not known. Therefore, Monte-Carlo estimate is utilised to compute integrals in Eq. (6) and Eq. (7). In practice that means the replacement of the integral by averaging over the batch or entire dataset.

2.2. Sequential neural eigenfunctions as networks

We represent each coordinate map ϕ_j by a scalar neural ansatz with parameters θ_j, chosen disjoint across j. At outer index k we optimise only θ_k; for every $j < k$ the maps enter the Gram and classification losses as *fixed* nonlinearities, so gradients with respect to θ_j vanish identically and the empirical Gram $\hat{C}_k$ is consistent with a block-coordinate descent on $(\theta_1, \ldots, \theta_K)$. The Dirichlet penalty involves $\nabla_{\mathbf{x}}\phi_k$ alone. The classifier logits append zeros in coordinates that have not yet been activated; that is, coordinates with index $j > k$ contribute zero to the classification loss throughout their development phase, which is equivalent to evaluating the affine readout on the leading k-dimensional coordinate subspace until the schedule completes.

We do not identify (ϕ_j) with eigenfunctions of a prescribed self-adjoint operator on $L^2(p)$; rather, they form orthogonal, Dirichlet-regularised coordinates biased by cross-entropy, analogously to learned spectral features (Deng et al., 2022).

The core innovation is the interplay between two learned components: (i) the BasisSet learns eigenfunctions $\phi_1, \ldots, \phi_K$ sequentially, with exactly one active at each outer iteration; (ii) the MetricNet learns $\mathbf{A}(\mathbf{x}) = \mathbf{\Lambda}(\mathbf{x}) \cdot \mathbf{U}_{\text{Trotter}}(\boldsymbol{\omega}(\mathbf{x}))$, where $\mathbf{\Lambda}$ is diagonal scaling and $\mathbf{U}$ is a product of Givens rotations. These two paths interact via the Modified Dirichlet Energy loss in (5), creating bidirectional feedback: as each eigenfunction's gradient is computed, it receives an update signal from $\mathbf{A}(\mathbf{x})$, and $\mathbf{A}(\mathbf{x})$ learns how to best scale and rotate the gradients for the current active eigenfunction. This synergy is essential to the method's stability and effectiveness. Figure 1 provides a high-level system schematic.

Algorithm 1 Sequential block-coordinate descent for EFDO.

for $k = 1$ to K **do**
  for each inner iteration until the budget is exhausted **do**
    Fix θ_j for all $j < k$.
    Evaluate $\Phi_{1:k}$ by applying $\phi_j(\cdot;\theta_j)$ with fixed θ_j for $j < k$ and the current iterate θ_k for $j = k$.
    Accumulate $\|\hat{C}_k - I_k\|_F^2$, the Dirichlet contribution $\|\mathbf{A}\nabla\phi_k\|^2$, and cross-entropy computed on logits whose coordinates with index $j > k$ are fixed to zero.
    Refresh $(w_{\text{orth}}, w_{\text{class}}, w_{\text{mde}})$ via (10) with $g_k = \|\hat{C}_k - I_k\|_F$, treating these weights as constants in the differentiation of $\mathcal{L}$.
  end for
end for

Algorithm 1 lists the sequential optimisation schedule, while Figure 3 presents the full training procedure in structured form. Figure 2 summarises the forward and backward coupling at outer index k; the narrative below matches the same notation. At outer index k, the frozen stack $\mathbf{\Phi}_{1:k-1}$ and the trainable map ϕ_k jointly feed $\mathcal{L}_{\text{orth}}$ and $\mathcal{L}_{\text{class}}$, while $\mathcal{L}_{\text{mde}}$ couples $\mathbf{A}(\mathbf{x})$ to $\nabla_{\mathbf{x}}\phi_k$ (see (5)); only θ_k receives gradients. The losses aggregate as in (9) with weights (10). After the schedule completes, inference uses the affine readout on the leading coordinates (8).

2.3. Loss-Function Weighting

In the present work we utilise the hierarchy of loss-functions:

- orthonormality
- cross-entropy for classification
- MDE loss-function

The objective is to learn representative features first and train the classifier with them. The final stage of MDE minimisation provides regularisation and improves the expressive power of constructed features.

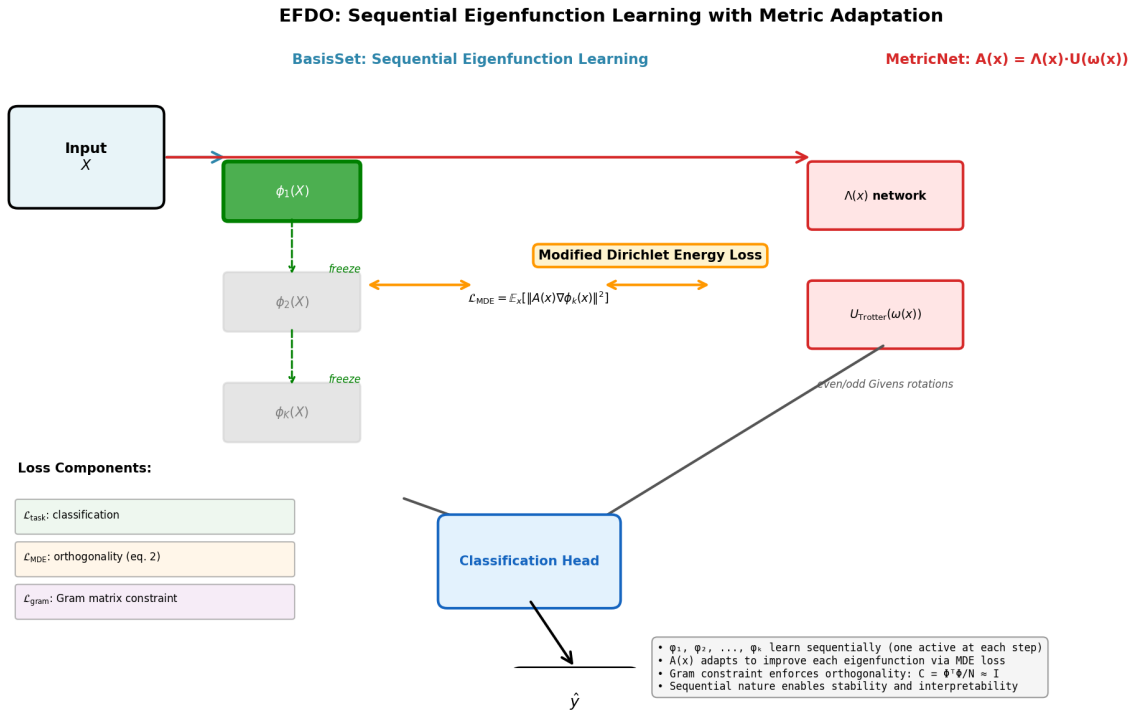

Figure 1. Overview of the EFDO framework. Data $\mathbf{X}$ flows through two interacting pathways: (left) BasisSet sequentially activates eigenfunctions $\phi_1, \phi_2, \ldots, \phi_K$, with only one trainable at each step; (right) MetricNet learns $\mathbf{A}(\mathbf{x}) = \mathbf{\Lambda}(\mathbf{x}) \cdot \mathbf{U}_{\text{Trotter}}(\boldsymbol{\omega}(\mathbf{x}))$. The two paths couple via the Modified Dirichlet Energy loss $\mathcal{L}_{\text{mde}} = \mathbb{E}_x[\|\mathbf{A}(\mathbf{x})\nabla\phi_k(\mathbf{x})\|^2]$, creating bidirectional feedback. The Classification-Head combines all eigenfunctions into a linear classifier. The three loss components (task, MDE, Gram-matrix constraint) are weighted dynamically as in Eq. (9).

Throughout all reported experiments, we fix two temperature scales: T_{orth}=0.1 and T_{class}=0.5. These values were selected via informal grid search on the MNIST validation set and held constant across all datasets; a formal hyperparameter ablation remains an open direction for future work.

The practical way to achieve the hierarchy of concern is dynamic weighting of loss-functions that is updated each iteration of loss-function minimisation: Denote the batch Gram Frobenius residual $g_k = \|\hat{C}_k - I_k\|_F$.

$$\begin{cases} w_{\text{orth}} \propto 1, \\ w_{\text{class}} \propto \exp\big(-g_k/T_{\text{orth}}\big), \\ w_{\text{mde}} \propto \exp\big(-\max\big(g_k/T_{\text{orth}},\ \mathcal{L}_{\text{class}}/T_{\text{class}}\big)\big). \end{cases} \tag{10}$$

Here T_{orth} and T_{class} are temperature-style scales fixed *a priori* for the reported runs (T_{class}=0.5); an optional graph-based warm-start could in principle set them from held-out logistic loss (Gomez-Chova et al., 2008), but we do not use that stage here.

Equation (10) employs exponential dampening of weights when loss-function values are large. The exponential weighting was chosen empirically; alternative functional forms such as $1/(1+g_k)$ or polynomial dampening were tested informally and found less robust in early experiments, though systematic ablation of this choice remains an open question for future work

It is clear from the Eq. (10) that the desired prioritisation of loss-function is achieved with such a weighting scheme: for instance, classifier is not trained at all if the value of g_k exceeds a certain value. Similar holds for $\mathcal{L}_{\text{mde}}$.

2.4. Matrix parametrisation and TROTTER-fixed rotations

Equation (5) is evaluated using the gradient of the active map ϕ_k only (Sec. 2.2). We summarise the parametrisations for $\mathbf{A}(\mathbf{x})$ that appear in Table 1.

At the continuum level one may factor

$$\mathbf{A}(\mathbf{x}) = \mathbf{\Lambda}(\mathbf{x})\mathbf{U}(\mathbf{x}), \tag{11}$$

with diagonal $\mathbf{\Lambda}$ and orthogonal $\mathbf{U}$. **Diagonal scaling.** The log-eigenvalues follow a zero-mean MLP head so that

$$\sum_{i=1}^{d} \log \lambda_i(\mathbf{x}) = 0 \quad \Rightarrow \quad \det \mathbf{\Lambda}(\mathbf{x}) = 1. \tag{12}$$

Rotational factors could in principle be approximated by PINN-style surrogates (Lagaris et al., 1997); all rotation–

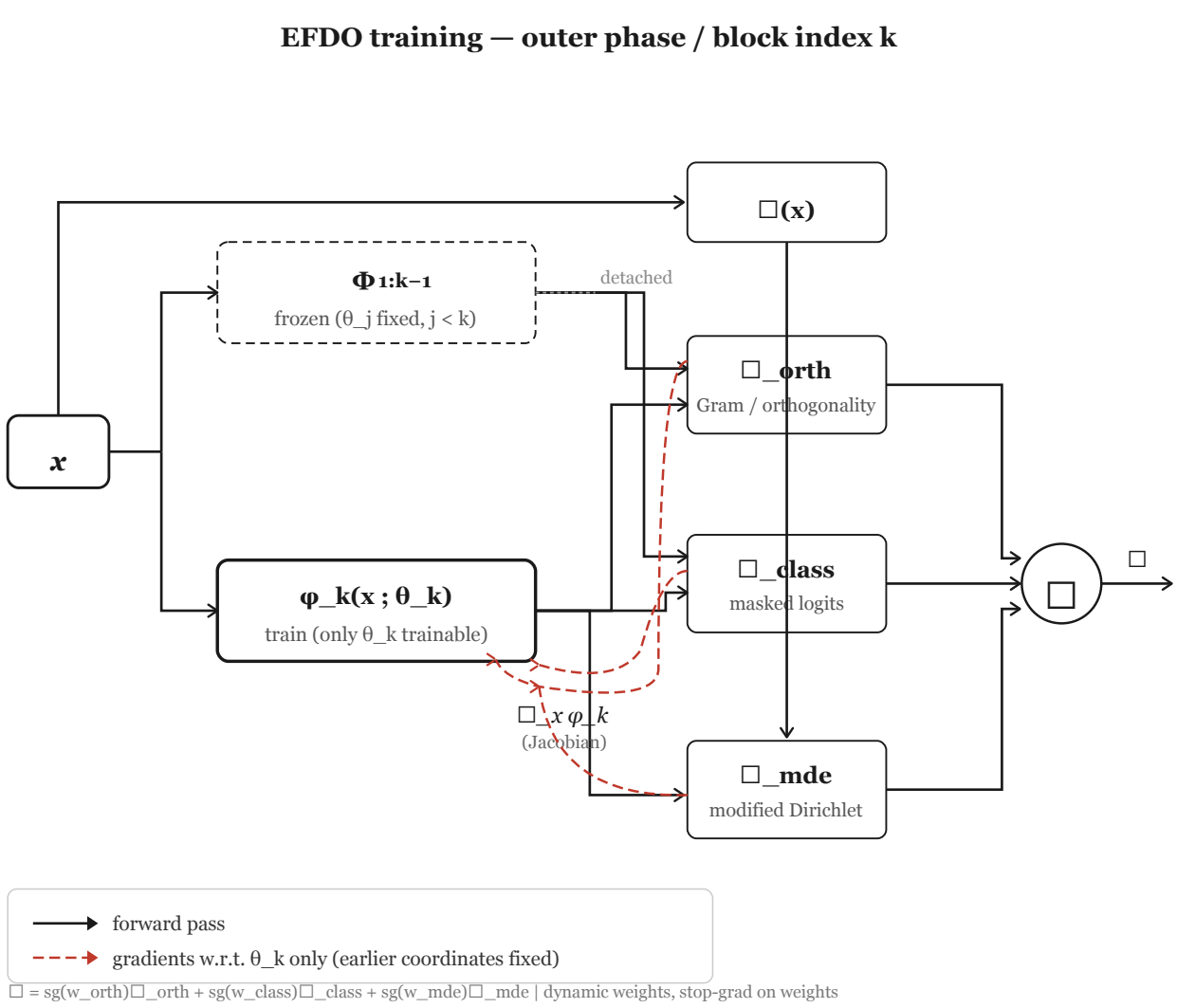

Figure 2. At outer index k, the stack $\mathbf{\Phi}_{1:k-1}$ is fixed while only $\phi_k(\cdot;\theta_k)$ is updated. Solid arrows denote forward evaluation; dashed red arrows denote gradients that update θ_k only (coordinates with $j < k$ are detached). The three losses combine as in (9) with weights from (10) and stop-gradient on $(w_{\text{orth}}, w_{\text{class}}, w_{\text{mde}})$. After training, logits follow $\mathbf{W\Phi} + \mathbf{b}$ on the leading coordinates (8).

scaling results in Table 1 use the learnt Givens parameterisation below.

Trained metric variants. **Metric disabled (OFF).** We set $\mathbf{A} = \mathbf{I}$ identically while still learning orthogonal features; Euclidean Dirichlet energies arise as the special case of (5).

Diagonal (DIAG). $\mathbf{A}(\mathbf{x}) = \mathbf{\Lambda}(\mathbf{x})$ with coordinates $\lambda_i(\mathbf{x}) = \exp(z_i(\mathbf{x}) - \bar{z}(\mathbf{x}))$ under (12).

Trotter rotation (TROTTER). We parameterise $\mathbf{U}(\mathbf{x})$ as a product of $d-1$ Givens rotations,

$$\mathbf{U}(\omega) = R_{d-2}(\omega_{d-2}) \cdots R_1(\omega_1)\, R_0(\omega_0), \qquad (13)$$

with angles $\omega_i(\mathbf{x}) = \pi \tanh\big(\mathrm{MLP}(\mathbf{x})_i\big)$ restricting $\mathbf{U}(\mathbf{x})$ to $\mathrm{SO}(d)$. Note: while Givens rotations form a proper subgroup of $\mathrm{SO}(d)$ with dimension $d-1$, bounding angles via $\omega_i = \pi \tanh(\cdot)$ keeps them in $[-\pi, \pi]$, ensuring numerical stability and preventing unbounded extrapolation in the angle parameterization. The learnt metric acts on tangent vectors $\mathbf{v}$—in practice $\mathbf{v} = \nabla\phi_k$—by rotating first and rescaling afterwards,

$$\mathbf{A}(\mathbf{x})\mathbf{v} = \mathbf{\Lambda}(\mathbf{x})\big(\mathbf{U}(\mathbf{x})\mathbf{v}\big), \qquad (14)$$

which we adopt throughout this work. **Gradient flow.** Equation (14) exposes both factors to stochastic gradients via $\|\mathbf{A}(\mathbf{x})\mathbf{v}\|^2 = \sum_i \lambda_i^2 \big((\mathbf{U}\mathbf{v})_i\big)^2$. If $\mathbf{\Lambda}$ is applied before $\mathbf{U}$, the Euclidean energy $\|\mathbf{A}\mathbf{v}\|^2$ collapses to a diagonal-only functional and no longer differentiates through the rotational factor; all experiments below therefore adopt (14). **Network architecture.** All basis functions and metric networks use three fully-connected hidden layers of width 64 with ReLU

Algorithm: Sequential Eigenfunction Dirichlet Optimization (EFDO)

```
Input: Dataset {(x₁, y₁)}, K ∈ ℕ (number of eigenfunctions), w_task, w_mde, w_gram (loss weights)

INITIALIZATION:
  • Initialize BasisSet with K learnable functions φ₁, φ₂, …, φₖ
  • Initialize MetricNet: A(x) = Λ(x) · U_Trotter(ω(x))  [exact orthogonal rotation]

FOR step t ← 1 to T_max DO:

  k ← ((t - 1) mod K) + 1    // Cycle through active eigenfunction: 1 → 2 → ⋯ → K → 1 → ⋯

  FOR each batch (X, y):

    FORWARD PASS:
      Φ(X) ← [φ₁(X), φ₂(X), …, φₖ(X)]ᵀ  // Evaluate all K eigenfunctions  (N × K)
      A(X) ← MetricNet(X)                 // Get adaptive metric             (N × d × d)
      ∇φₖ ← autodiff(φₖ(X))               // Gradient of ACTIVE eigenfunction (N × d)
      ŷ ← Head(Φ(X))                      // Linear classifier output        (N,)

    LOSS COMPUTATION:
      L_task  ← CrossEntropy(ŷ, y)                    // Classification loss
      L_mde   ← □ₓ[‖A(x)·∇φₖ(x)‖²]                    // Modified Dirichlet Energy
      L_gram  ← ‖Φ(X)ᵀΦ(X)/N - I‖²_F                  // Gram matrix constraint
      L_total ← w_task·L_task + w_mde·L_mde + w_gram·L_gram

    OPTIMIZATION:
      Backward: ∇_θₖ L_total, ∇_θₐ L_total          // Only active function & metric get gradients
      Update:   θₖ ← θₖ - α·∇_θₖ L_total             // Update active φₖ
                θₐ ← θₐ - α·∇_θₐ L_total             // Update metric A(x)
                // φ₁, …, φₖ₋₁, φₖ₊₁, …, φₖ stay FROZEN
```

Key Properties: • Sequential activation (k cycles through {1,…,K}) prevents basis collapse
• MDE loss couples metric A(x) with gradients ∇φₖ, creating synergy • Gram constraint enforces orthogonality C ≈ I

Figure 3. Detailed pseudocode for the Sequential Eigenfunction Dirichlet Optimization algorithm. The key insight is the cycling of the active eigenfunction $k \leftarrow ((t-1) \bmod K) + 1$: at each step, only one eigenfunction ϕ_k and the metric network $\mathbf{A}(\mathbf{x})$ receive gradient updates, while all others remain frozen. The three loss terms (classification, Modified Dirichlet Energy, and Gram-matrix orthogonality constraint) are computed jointly and combined with dynamic weights. This sequential activation prevents basis collapse, ensures stable convergence, and enables interpretability through individual eigenfunction monitoring.

activations. Basis functions ϕ_k output a single scalar; metric networks output K values (for the diagonal variant) or $(d-1) \times n_{\text{passes}}$ angle values (for the Trotter variant).

Numerical contraction. Adjacent plane rotations are fused into even–odd sweeps with depth bounded independently of the nominal Givens count, following standard practice for compact differentiable orthogonal maps.

EFDO-TROTTER on Two Moons, Circles, and HTRU2 aggregates five independent optimisation trajectories, each with 60,000 gradient steps, under (14). The same five-seed protocol applies to MNIST multiclass TROTTER at 60,000 steps, whereas CIFAR-10 ResNet-18 embeddings use 120,000 total steps per seed when TROTTER is enabled.

2.5. Pretraining.

Optional Graph-Laplacian pretraining could fit eigenmaps on a subsample and calibrate T_{class} from a linear probe (Gomez-Chova et al., 2008). **All experiments below** disable this stage, fix T_{class}=0.5, and train EFDO from scratch without graph-based warm starts.

3. Experimental Results

3.1. Datasets and Evaluation Protocol

We evaluate on **five** datasets spanning binary classification and multiclass pretrained embeddings.

Binary datasets. *Two Moons* and *Circles* are planar synthetic benchmarks with n=10,000 samples, noise σ=0.1, and a 70%/15%/15% split between training, validation,

and test data (Pedregosa et al., 2011). *HTRU2* (Lyon et al., 2016; Lyon, 2016) provides eight tabular features for pulsar candidate classification on 17,898 samples with the same split and z-scored inputs.

Multiclass and embedding benchmarks. *MNIST* (LeCun & Cortes, 2010) is used at native resolution with 784 input dimensions; we use the standard 60k training and 10k test split, further subdividing the test set into 5k validation and 5k held-out evaluation; we set $K{=}16$. *CIFAR-10* (Krizhevsky, 2009) is represented by 512-dimensional penultimate activations of a pretrained ResNet-18 using the standard 45k training, 5k validation, and 10k test partition; we set $K{=}16$.

Protocols. Unless noted otherwise, Table 1 reports EFDO **validation** accuracy with mean and standard deviation over five random seeds. NeuralEF entries marked in the table are **test** accuracy from the original benchmark report. We do not apply data augmentation or graph-Laplacian pretraining in the reported runs. Both basis and metric networks use three hidden layers of width 64; metric heads use two layers of width 64.

3.2. Baselines

We compare against two families of baselines.

Classical baselines. Random forests use 200 trees fit on raw inputs. Multinomial and binary logistic models use ℓ_2-regularised maximum likelihood with the default optimisation settings from the scikit-learn reference implementation (Pedregosa et al., 2011), evaluated on the official multiclass test split when it exists and otherwise on the validation split reserved for the low-dimensional benchmarks. Table 1 lists the resulting scores; missing entries are left blank when the corresponding baseline has not been recomputed on that split.

NeuralEF (Deng et al., 2022) is included at $K{=}16$ using the published MNIST test accuracy with mean and standard deviation over five seeds. We do not yet include a NeuralEF row for CIFAR-10 embeddings because the evaluation protocol is still being aligned with ours.

3.3. Main Results

Table 1 reports validation accuracy for all methods and datasets. The three EFDO variants are: **off** ($\mathbf{A}{=}\mathbf{I}$), **diag** ($\mathbf{A}{=}\mathbf{\Lambda}$), and **trotter** (full $\mathbf{A}{=}\mathbf{\Lambda U}$ with the *Trotter-fixed* apply order from Sec. 2.4). EFDO-TROTTER on Two Moons, Circles, and HTRU2 reports the mean over five random initialisations after 60,000 gradient steps satisfying (14). On MNIST multiclass we likewise aggregate five seeds after 60,000 total steps (Trotter-fixed metric, (14)). The dagger marks the CIFAR-10 embedding row only, where the five-seed TROTTER grid under 120,000 steps is not yet present in the synced logs.

How to read the results. On the *linearly separable* HTRU2 dataset, both LR and EFDO-off match RF, confirming the method does not degrade when the raw features are already informative. On *Two Moons*, where LR achieves only 87.8% (the linear boundary cannot separate the two crescents), EFDO-off reaches 100.0%, demonstrating that spectral features capture the nonlinear structure of the data manifold. This perfect accuracy with zero variance across seeds is possible because the crescent geometry admits an exact nonlinear separation via the learned eigenfunction ϕ_2, and the validation set contains no pathological cases that would prevent consistent convergence across random initializations. On *MNIST*, EFDO-off reaches 94.53% validation accuracy. The cited NeuralEF score is 82.52% test accuracy at the same feature dimension $K{=}16$; the metrics are not directly comparable, but the gap motivates aligned test-time evaluation in future work. NeuralEF numbers for CIFAR-10 embeddings are omitted for the same reason. The *Circles* dataset exhibits substantially higher variance across seeds (e.g., 14.9% standard deviation for EFDO-off) compared to other benchmarks. This instability arises because the concentric-ring topology creates a difficult landscape for gradient-based optimization: the decision boundary must separate inner and outer rings, requiring eigenfunctions with specific radial sensitivity. Some random initializations discover good radial structure early, while others get trapped in suboptimal solutions that separate points by angular patterns instead. Notably, the diagonal variant (EFDO-diag) shows lower variance (4.8%), suggesting that per-coordinate scaling helps stabilize learning on this topology. With full rotation–scaling (TROTTER), the mean validation accuracy improves to 83.59% under the identical 60,000-step scheduling. While the improvement from EFDO-off (78.2%) to EFDO-trotter (83.6%) spans 5.4 percentage points, the overlapping error bars (±14.9% and ±15.7% respectively) suggest modest statistical separation, warranting larger-sample confirmation in future work. On CIFAR-10 embeddings, EFDO-diag trails EFDO-off slightly, underscoring that extra scaling does not always help high-level features.

3.4. Eigenfunction Visualisation

Figure 4 shows the first three learned eigenfunctions ϕ_1, ϕ_2, ϕ_3 on Two Moons and Circles (EFDO-off, representative replicate). The maps capture nonlinear structure: on Two Moons, ϕ_2 and ϕ_3 align more clearly with class geometry, whereas ϕ_1 behaves almost constant on validation (see Fig. 7 for separators in (ϕ_2, ϕ_3)-space). This confirms that the minimisation of the Modified Dirichlet Energy produces representations that are geometrically meaningful rather than purely discriminative. We retain two views: Fig. 4 highlights each coordinate on labelled points, while Fig. 7

Table 1. Validation accuracy in percent, mean ± standard deviation over five random initialisations. Bold entries mark the best EFDO variant on each dataset. TROTTER rows for Two Moons, Circles, and HTRU2 follow (14) with 60,000 gradient updates per run; MNIST TROTTER uses the same schedule. Classical baselines use the configuration described in Sec. 3.1. † marks CIFAR-10 TROTTER on ResNet-18 embeddings still pending in the experiment export. * cites the published NeuralEF MNIST *test* accuracy.

Dataset	RF	LR	NeuralEF*	EFDO-off	EFDO-diag	EFDO-trotter
Two Moons	99.77±0.04	87.80±0.00	—	**100.00**±0.00	99.97±0.04	99.99±0.03
Circles	98.53±0.11	50.40±0.00	—	78.23±14.90	79.16±4.82	**83.59**±15.70
HTRU2	98.07±0.07	98.14±0.00	—	97.52±0.08	97.48±0.04	**97.71**±0.06
MNIST	97.12±0.03	91.84±0.00	82.52±0.29	**94.53**±0.33	93.63±0.34	93.99±0.25
CIFAR-10	—	—	—	**85.50**±0.53	84.98±0.33	†

couples (ϕ_2, ϕ_3) with a logistic separator and its back-map to (x_1, x_2), which is not readable from the scalar panels alone.

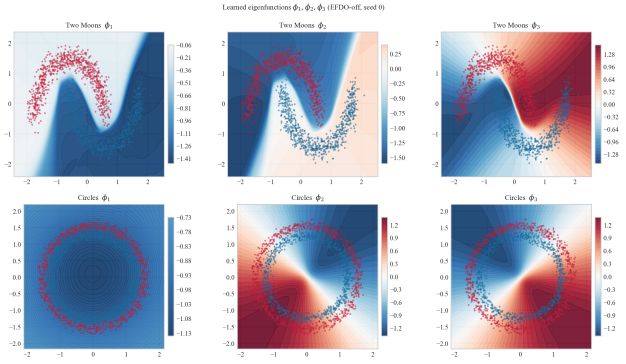

Figure 4. First three learned maps ϕ_1, ϕ_2, ϕ_3 on Two Moons and Circles for EFDO-off on a representative replicate. Point colour encodes class labels; the background encodes map values.

3.5. Training Dynamics

Figure 5 shows validation accuracy over training steps for MNIST and CIFAR-10 (EFDO-off and EFDO-diag, mean ± std over 5 seeds). Both variants converge rapidly in the first 15k steps and remain stable thereafter, with negligible variance across seeds. The diagonal variant is slightly weaker on MNIST but competitive on ResNet embeddings, reflecting different inductive biases between raw pixels and deep features.

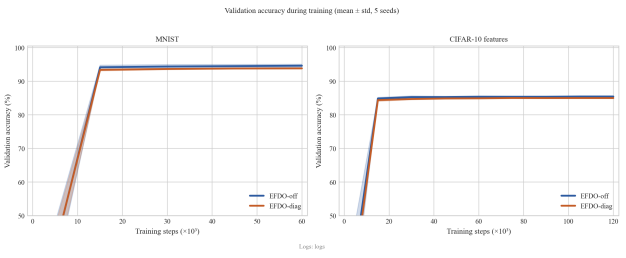

Figure 5. Validation accuracy during training, mean ± standard deviation over five seeds. MNIST appears on the left and CIFAR-10 ResNet-18 embeddings on the right.

3.6. Gram Matrix Analysis

Figure 6 shows the Gram matrix $C_K = \mathbb{E}[\Phi\Phi^\top]$ computed on the MNIST validation set after training (EFDO-off, K=16, representative replicate). The matrix is close to the identity with moderate Frobenius deviation $\|C-I\|_F$, consistent with the batch Gram surrogate in Remark 2.1.

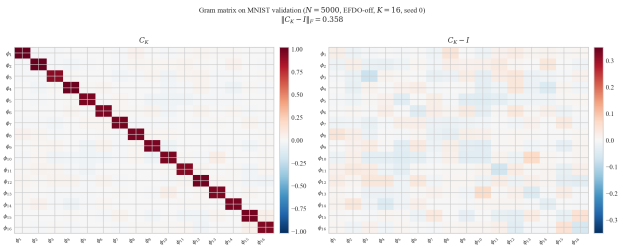

Figure 6. Empirical Gram matrix C_K and deviation $C_K - I$ on the MNIST validation set for EFDO-off with K=16 on a representative replicate. The annotation reports $\|C_K - I\|_F$ on the same validation draw.

Figure 7 shows how nonlinear coordinates reshape class boundaries in input space.

Gradients were obtained by automatic differentiation (Paszke et al., 2019) and optimisation used mini-batch stochastic gradient descent with the budgets stated above; hardware choices affect wall-clock time but not the reported accuracies. Approximation of a single eigen-function takes about two hours on commodity CPUs at the quoted mesh resolution, while the sequential construction on MNIST at d=784 requires a few minutes per coordinate when the same iteration budget is executed on a modern accelerator. Therefore, computational time is one of the most essential disadvantages of the proposed approach on CPU. The solution to that problem could be appropriate initialization of the neural network, but it is the subject of future research.

Finally, it is important to notice that the warm-up stage is critical to avoiding local-minimum traps. The intuition behind the weighting introduced is based on the observation that the Dirichlet Energy is high for strongly oscillating functions. At the same moment, highly oscillating functions are likely to have non-zero projection on the linear subspace

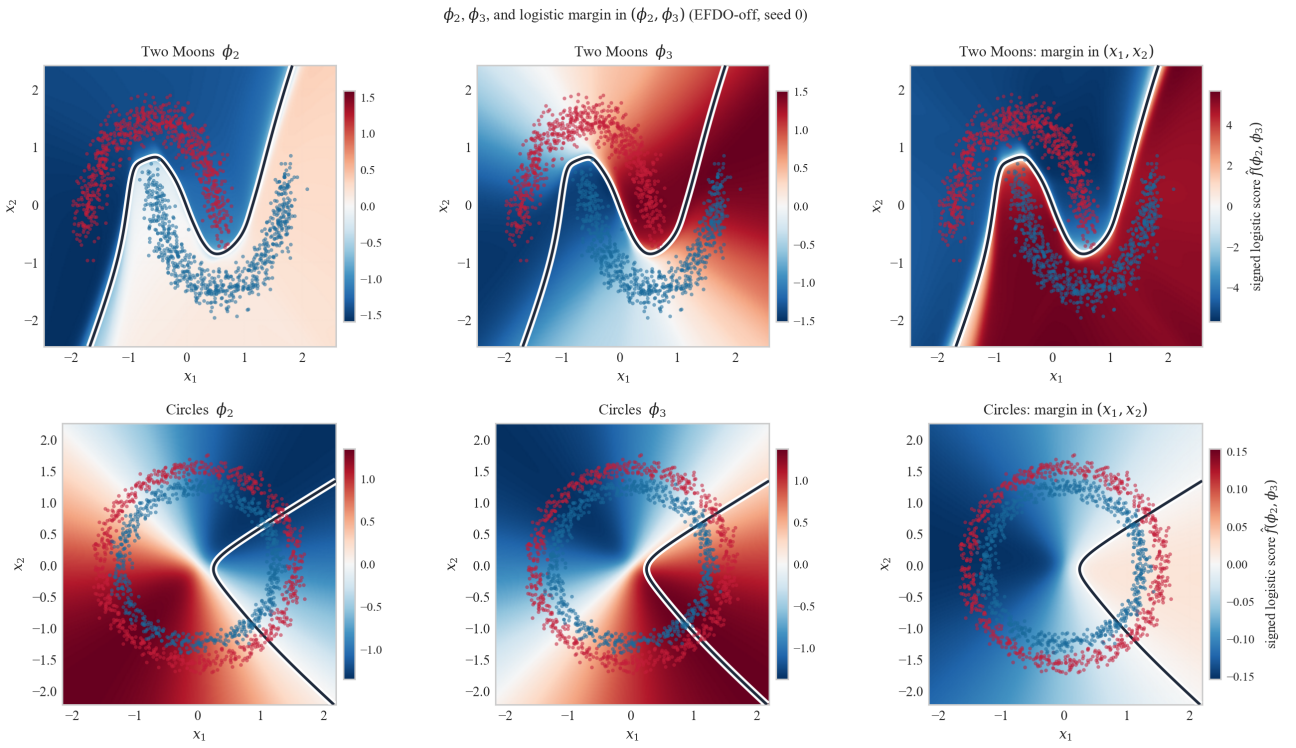

Figure 7. Maps ϕ_2 and ϕ_3 on dense (x_1, x_2) grids for Two Moons (top) and Circles (bottom) under EFDO-off on a representative replicate. The first two columns show each map; validation points are overlaid with the zero contour of a logistic decision function in (ϕ_2, ϕ_3). The third column colours the signed margin lifted to input space.

of functions orthogonal to smooth eigenfunctions that have been approximated already. Therefore, by promoting oscillations and orthogonality at the beginning of training, we can increase the capability of ANN for detection of orthogonal directions in a function space.

4. Discussion and Concluding Remarks.

In the current work we present the novel sequential approach for approximation of Laplacian-like differential operators. The central idea of the method is a two-stage procedure that allows the ANN to escape from the local minimum and find the direction orthogonal to the space spanned by previously constructed eigenfunctions. We achieve that by introducing the instability to the optimization process that deforms the ANN approximation of eigen function to such a degree that novel direction can be discovered.

This instability stage or warm-up stage is followed by optimization of the loss function with positive weights with a certain hierarchy. That weights are constructed in such a way that ANN tries to satisfy orthogonalization and normalization constraints first before minimizing the objective function, which is the Dirichlet Energy. This idea is quite generic and can be directly applied to training of other ANNs with constraints.

In addition to the training procedure above, small-scale synthetic checks illustrate how oversized networks can violate discrete spectral lower bounds; we leave principled augmentation of the coordinate maps to future work and do not use augmentation in Table 1.

Finally, on the benchmarks of Sec. 3, models trained on (ϕ_j) achieve validation accuracy competitive with strong baselines on raw inputs or embeddings, supporting EFDO as a lightweight inference stack once the maps are trained. One of the main practical drawbacks remains optimisation time on CPU; better initialisation is left for future work.

Limitations. EFDO learns task-dependent coordinates rather than eigenmodes of a fixed differential operator independent of supervision. Finite-batch Gram penalties only approximate global L^2-orthogonality, and masking inactive logits couples the outer coordinate schedule to the classifier geometry (Deng et al., 2022). The adaptive weights in (10) inherit Monte Carlo noise through g_k. Comparisons with NeuralEF on CIFAR-10 embeddings remain open, as does the embedding-space multiclass TROTTER row once all five long runs finish.

References

Belkin, M. and Niyogi, P. Towards a theoretical foundation for laplacian-based manifold methods. *Journal of Computer and System Sciences*, 74(8):1289–1308, 2008. ISSN 0022-0000. doi: https://doi.org/10.1016/j.jcss.2007.08.006. URL https://www.sciencedirect.com/science/article/pii/S0022000007001274. Learning Theory 2005.

Choo, K., Mezzacapo, A., and Carleo, G. Fermionic neural-network states for ab-initio electronic structure. *Nature Communications*, 11, 05 2020. doi: 10.1038/s41467-020-15724-9.

Deng, Z., Shi, J., and Zhu, J. NeuralEF: Deconstructing kernels by deep neural networks. In Chaudhuri, K., Jegelka, S., Song, L., Szepesvari, C., Niu, G., and Sabato, S. (eds.), *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pp. 4976–4992. PMLR, 17–23 Jul 2022. URL https://proceedings.mlr.press/v162/deng22b.html.

Evans, L. C. *Partial differential equations*, volume 19. American Mathematical Society, 2022.

Feld, T., Aujol, J.-F., Gilboa, G., and Papadakis, N. Rayleigh quotient minimization for absolutely one-homogeneous functionals. *Inverse Problems*, 35(6):064003, may 2019. doi: 10.1088/1361-6420/ab0cb2. URL https://dx.doi.org/10.1088/1361-6420/ab0cb2.

Ford, W. Chapter 5 - eigenvalues and eigenvectors. In Ford, W. (ed.), *Numerical Linear Algebra with Applications*, pp. 79–101. Academic Press, Boston, 2015. ISBN 978-0-12-394435-1. doi: https://doi.org/10.1016/B978-0-12-394435-1.00005-3. URL https://www.sciencedirect.com/science/article/pii/B9780123944351000053.

Gomez-Chova, L., Camps-Valls, G., Munoz-Mari, J., and Calpe, J. Semisupervised image classification with laplacian support vector machines. *IEEE Geoscience and Remote Sensing Letters*, 5(3):336–340, 2008. doi: 10.1109/LGRS.2008.916070.

Jin, H., Mattheakis, M., and Protopapas, P. Unsupervised neural networks for quantum eigenvalue problems. *ArXiv*, abs/2010.05075, 2020. URL https://api.semanticscholar.org/CorpusID:222290530.

Krizhevsky, A. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009. URL https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf.

Kunegis, J., Schmidt, S., Lommatzsch, A., Lerner, J., Luca, E. W. D., and Albayrak, S. *Spectral Analysis of Signed Graphs for Clustering, Prediction and Visualization*, pp. 559–570. doi: 10.1137/1.9781611972801.49. URL https://epubs.siam.org/doi/abs/10.1137/1.9781611972801.49.

Lagaris, I., Likas, A., and Fotiadis, D. Artificial neural network methods in quantum mechanics. *Computer Physics Communications*, 104(1):1–14, 1997. ISSN 0010-4655. doi: https://doi.org/10.1016/S0010-4655(97)00054-4. URL https://www.sciencedirect.com/science/article/pii/S0010465597000544.

LeCun, Y. and Cortes, C. MNIST handwritten digit database. 2010. URL http://yann.lecun.com/exdb/mnist/.

Lyon, R. HTRU2. 3 2016. doi: 10.6084/m9.figshare.3080389.v1. URL https://figshare.com/articles/dataset/HTRU2/3080389.

Lyon, R. J., Stappers, B. W., Cooper, S., Brooke, J. M., and Knowles, J. D. Fifty years of pulsar candidate selection: from simple filters to a new principled real-time classification approach. *Monthly Notices of the Royal Astronomical Society*, 459(1):1104–1123, 04 2016. ISSN 0035-8711. doi: 10.1093/mnras/stw656. URL https://doi.org/10.1093/mnras/stw656.

Ng, A., Jordan, M., and Weiss, Y. On spectral clustering: Analysis and an algorithm. In Dietterich, T., Becker, S., and Ghahramani, Z. (eds.), *Advances in Neural Information Processing Systems*, volume 14. MIT Press, 2001. URL https://proceedings.neurips.cc/paper_files/paper/2001/file/801272ee79cfde7fa5960571fee36b9b-Paper.pdf.

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., and Chintala, S. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32*, pp. 8024–8035. Curran Associates, Inc., 2019. URL http://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance.pdf.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., and Duchesnay, E. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.